

Facade Design Pattern

Simplifying Complex Systems

Md Mehedi Hasan Maruf-2003037 Md Shahidul Islam Dipu-2003040
Eazdan Mostafa Rafin-2003055

Department of Computer Science and Engineering, RUET

January 7, 2025

What is a Design Pattern?

- Reusable solutions to common software design problems
- Not code, but templates for solving problems
- Enhance code maintainability and scalability

Types of Design Patterns:

- **Creational Patterns**
- **Structural Patterns**
- **Behavioral Patterns**

Facade Design Pattern

- **Type:** Structural Design Pattern
- **Objective:**
 - Provide a simplified interface to complex subsystems.
 - Reduce dependencies between the client and subsystem classes.

When to Use Facade

- Complex Subsystems with many interdependent classes
- Need to provide a simple interface for clients
- Layered systems requiring interaction with lower layers
- Integrating with legacy or third-party code

Key Components

- ① **Facade:** Simplified interface for the client
- ② **Subsystem Classes:** Complex classes handling specific functionalities
- ③ **Client:** Interacts only with the Facade

Case Study: Online Exam Proctoring System

Problem Statement

Design a system to monitor students during online examinations to prevent cheating and ensure exam integrity. The system should handle:

- Real-time webcam monitoring
- Screen recording
- Cheating detection

Requirements

- Monitor student's webcam feed
- Record screen activities
- Detect suspicious behavior
- Log all activities
- Handle multiple monitoring tasks concurrently

System Components

Component	Description
ProctorFacade	Main interface that coordinates all monitoring activities
WebcamMonitor	Handles webcam feed capture and processing
ScreenRecorder	Captures and saves screen activities
CheatingDetector	Analyzes feeds for suspicious behavior
Logger	Manages system-wide logging and activity tracking

ProctorFacade Class: Initialization

```
1 class ProctorFacade:
2     def __init__(self):
3         self.webcam = WebcamMonitor()
4         self.screen_recorder = ScreenRecorder()
5         self.cheating_detector = CheatingDetector()
6         self.logger = Logger().logger
7
8     def start_proctoring(self, duration=300):
9         """
10        Start proctoring for a specific duration in seconds.
11        """
12        self.logger.info(f"Starting Online Exam Proctoring - Duration: {duration} seconds")
```

ProctorFacade Class: Proctoring Implementation

```
1  try:
2      # Start all components
3      webcam_thread = threading.Thread(target=self.webcam.start_capture, args=(duration,))
4      screen_thread = threading.Thread(target=self.screen_recorder.start_recording,
5      ↪ args=(duration,))
6      cheat_thread = threading.Thread(target=self.cheating_detector.detect_cheating,
7      ↪ args=(duration,))
8
9      webcam_thread.start()
10     self.logger.info("Webcam monitoring started")
11     screen_thread.start()
12     self.logger.info("Screen recording started")
13     cheat_thread.start()
14     self.logger.info("Cheating detection system started")
15
16     # Wait for all threads to complete
17     webcam_thread.join()
18     screen_thread.join()
19     cheat_thread.join()
20
21     self.logger.info("Proctoring session completed successfully")
22 except Exception as e:
23     self.logger.error(f"Error during proctoring: {str(e)}")
24     raise
```

Subsystem Interfaces

```
1 class WebcamMonitor:
2     def start_capture(self, duration):
3         """Capture webcam feed for a specific duration."""
4         pass
5
6 class ScreenRecorder:
7     def start_recording(self, duration):
8         """Record the screen for a specific duration."""
9         pass
10
11 class CheatingDetector:
12     def detect_cheating(self, duration):
13         """Detect cheating activities using AI."""
14         pass
```

System Logs

```
2025-01-07 01:08:18,800 - INFO - Starting Online Exam Proctoring - Duration:  
↪ 60 seconds  
2025-01-07 01:08:18,801 - INFO - Webcam monitoring started  
2025-01-07 01:08:18,802 - INFO - Duration: 60 seconds  
2025-01-07 01:08:18,802 - INFO - Screen recording started  
2025-01-07 01:08:18,803 - INFO - Screen recording started  
2025-01-07 01:08:18,803 - INFO - Webcam monitoring Ended Successfully  
2025-01-07 01:08:18,804 - INFO - Cheating detection system started  
2025-01-07 01:08:18,805 - WARNING - This is a mock implementation for  
↪ demonstration purposes only!  
2025-01-07 01:08:18,806 - INFO - Duration: 60 seconds  
2025-01-07 01:08:18,806 - INFO - Cheating detection system Ended Successfully  
2025-01-07 01:08:18,807 - INFO - Proctoring session completed successfully
```

Pros

- Simplifies interactions with complex systems
- Promotes loose coupling between client and subsystem
- Enhances code readability and maintainability
- Facilitates easier testing and debugging

- **Hides Too Much:** Can block access to advanced features.
- **Overdependence:** Makes it harder to work without the Facade.
- **Maintenance Overhead:** Needs regular updates to stay in sync.

Real-World Applications

- **Home Theater Systems:** One button dims lights, starts the projector, and plays your movie.
- **Computer Start-Up:** Turns chaos into a smooth boot-up process.
- **Online Shopping:** Handles payments, inventory, and shipping in the background.

Summary

The Facade Design Pattern simplifies complex systems by providing a clean and easy-to-use interface, making your code more readable, maintainable, and less dependent on the intricate details of subsystems.